The Pythoneers

Introduction to the Savitzky-Golay Filter: A Comprehensive Guide (Using Python)

Smoothing Data with Precision: Step-by-Step Applications and Visualizations in Python

Thomas Konstantinovsky Follow 8 min read · Jul 22, 2024

Introduction

In the realm of data analysis, particularly when dealing with noisy signals, smoothing techniques play a crucial role in extracting meaningful information. Whether you're working with experimental data, financial time series, or any form of signal processing, noise can blur the underlying patterns and trends you aim to study/uncover. Various methods exist to tackle this issue (some simpler and faster than others), but in my personal experience, when working with short signals and runtime, it can be overlooked in favor of quality; one technique stands out due to its ability to smooth data while preserving its important features — the **Savitzky-Golay filter.**

The Savitzky-Golay filter, developed by Abraham Savitzky and Marcel J. E. Golay in 1964, is a digital filter widely used for data smoothing and differentiation. Unlike many other smoothing methods, which can distort the signal by blurring its sharp features (take the median or mean filters as an example), the Savitzky-Golay filter excels in maintaining the integrity of the original signal. This makes it particularly useful in applications where preserving the shape and features of the signal is one of the constraints.

Here, I will take you on a comprehensive journey to understand the Savitzky-Golay filter. We will explore its underlying logic, delve into the mathematical formulation, and try to develop intuition for each step of the process. Additionally, we will visualize the filter's application using Python, providing a hands-on understanding. By the end of this post, I hope you will have a solid grasp of how the Savitzky-Golay filter works and how to apply it effectively in your data analysis adventures!

What is the Savitzky-Golay Filter?

The Savitzky-Golay filter is a digital filter that smooths data points by fitting successive sub-sets of adjacent data points with a low-degree polynomial using the method of linear least squares (also explained further in this post). Its primary advantage lies in its ability to smooth data while preserving the features of the original signal, such as peaks and valleys, which might be lost with other smoothing techniques (see other techniques 1,2,3,4.)

Definition and Basic Concept

The Savitzky-Golay filter works by sliding a window of fixed size (one of its hyperparameters) over the data and fitting a polynomial to the points within this window (the degree of the polynomial is also a hyperparameter). The value of the polynomial at the central point of the window is then taken as

the smoothed value. This process is repeated for each point in the data set, resulting in a smoothed signal.

Let us try and break down the process into bite-size pieces and understand each part individually.

Polynomial Fitting Using Least Squares

The core idea is to approximate the data points within a moving window by a polynomial of a certain degree. Suppose we have a set of data points (**xi**, **yi**) where **i** ranges from **1** to **N**. We aim to fit a polynomial of degree **p** to these points.

The polynomial can be expressed as:

$$y = a_0 + a_1 x + a_2 x^2 + \cdots + a_p x^p$$

polynomial of degree **p**

For a given window of data points centered at **x_k**, we need to determine the coefficients **a0**, **a1** , **... ,ap** such that the polynomial best fits the data points within the window. This is achieved by minimizing the sum of the squares of

the differences between the actual data points **y_i** and the polynomial values **y_ih**:

$$\text{Minimize} \sum_{i=-m}^{m} \left(y_{k+i} - \sum_{j=0}^{p} a_j x_{k+i}^j \right)^2$$

minimizing the sum of the squares of the differences between the actual data points **y_i** and the polynomial values **y_ih.** Here, **2m+1** is the window size centered around the point **x_k**.

Example of Polynomial Fitting

Lets make sure we have understood the above concept as it is fundamental for our next steps!
Consider a simple example with a window size of 5 (i.e., **m=2**) and a polynomial of degree **2 (p)**. Suppose we have the following data points within the window:

$$(x_{k-2}, y_{k-2}), (x_{k-1}, y_{k-1}), (x_k, y_k), (x_{k+1}, y_{k+1}), (x_{k+2}, y_{k+2})$$

the 5 data points within the window centered at **x_k**

The polynomial we fit to these points is:

$$y = a_0 + a_1 x + a_2 x^2$$

the polynomial of degree 2

We solve for **a0**, **a1**, **a2** by minimizing the sum of squared errors:

$$\text{Minimize} \sum_{i=-2}^{2} \left(y_{k+i} - (a_0 + a_1 x_{k+i} + a_2 x_{k+i}^2)\right)^2$$

implementing this from scratch), the smoothed value at **x_k** is given by the polynomial evaluated at **x_k**:

$$\hat{y}_k = a_0 + a_1 x_k + a_2 x_k^2$$

where **yhat_k** is the smoothed value at **x_k**

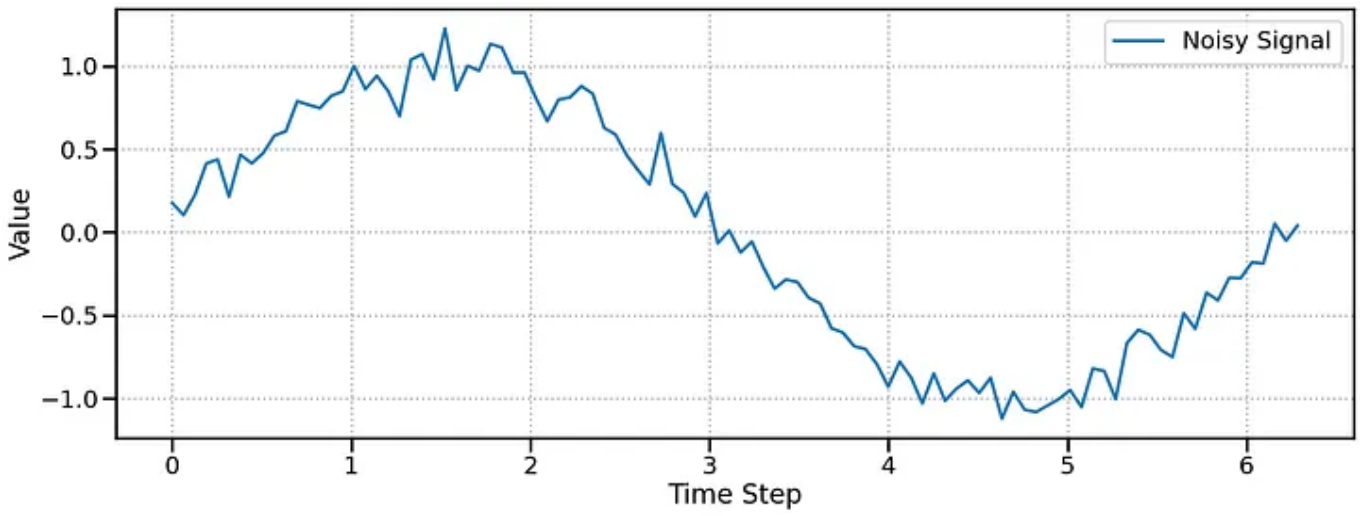
Step-by-Step Example:

Let's go through a detailed example to see the Savitzky-Golay filter in action!

Generating a Noisy Signal

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter

np.random.seed(0)
x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x) + np.random.normal(0, 0.1, x.size)

plt.plot(x, y, label='Noisy Signal')
plt.grid(lw=2,ls=':')
```

```
plt.xlabel('Time Step')
plt.ylabel("Value")
plt.legend()
plt.show()
```

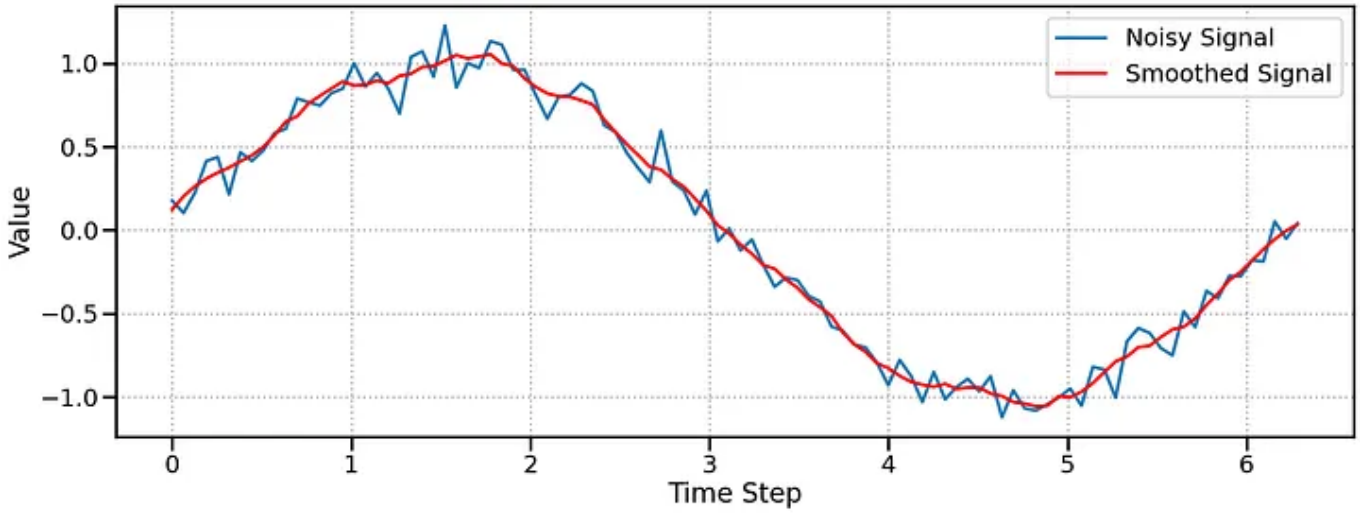


The Noisy Example Signal

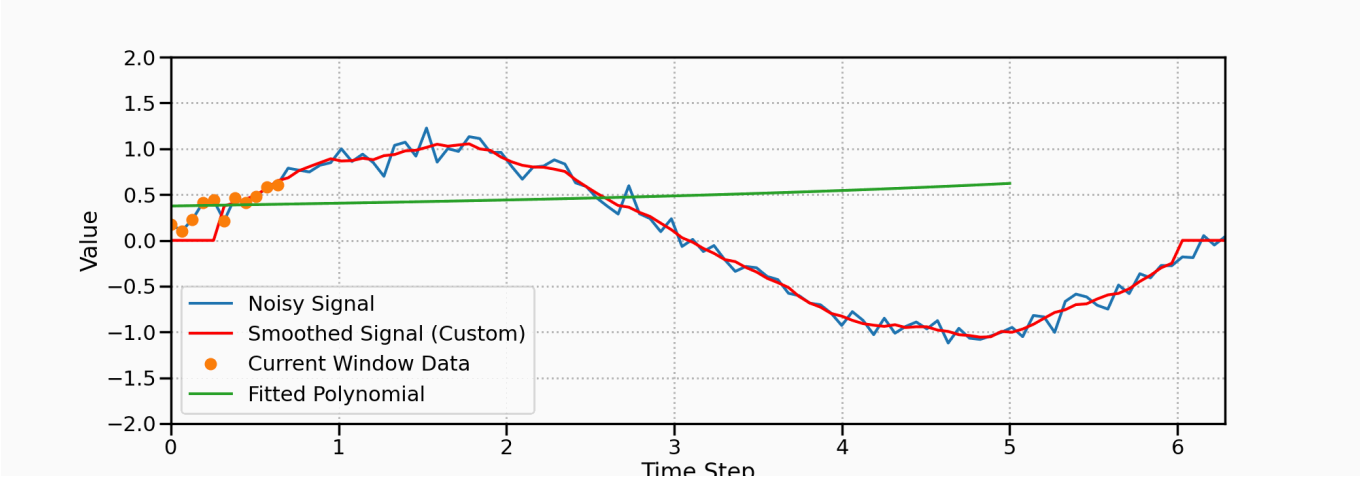
Applying the Savitzky-Golay Filter

We choose a window size of 11 and a polynomial degree of 3.

```
window_size = 11
poly_order = 3
y_smooth = savgol_filter(y, window_size, poly_order)

plt.plot(x, y, label='Noisy Signal')
plt.plot(x, y_smooth, label='Smoothed Signal', color='red')
plt.grid(lw=2,ls=':')
plt.xlabel('Time Step')
plt.ylabel("Value")
plt.legend()
plt.show()
```



The resulting signal after applying the savgol filter



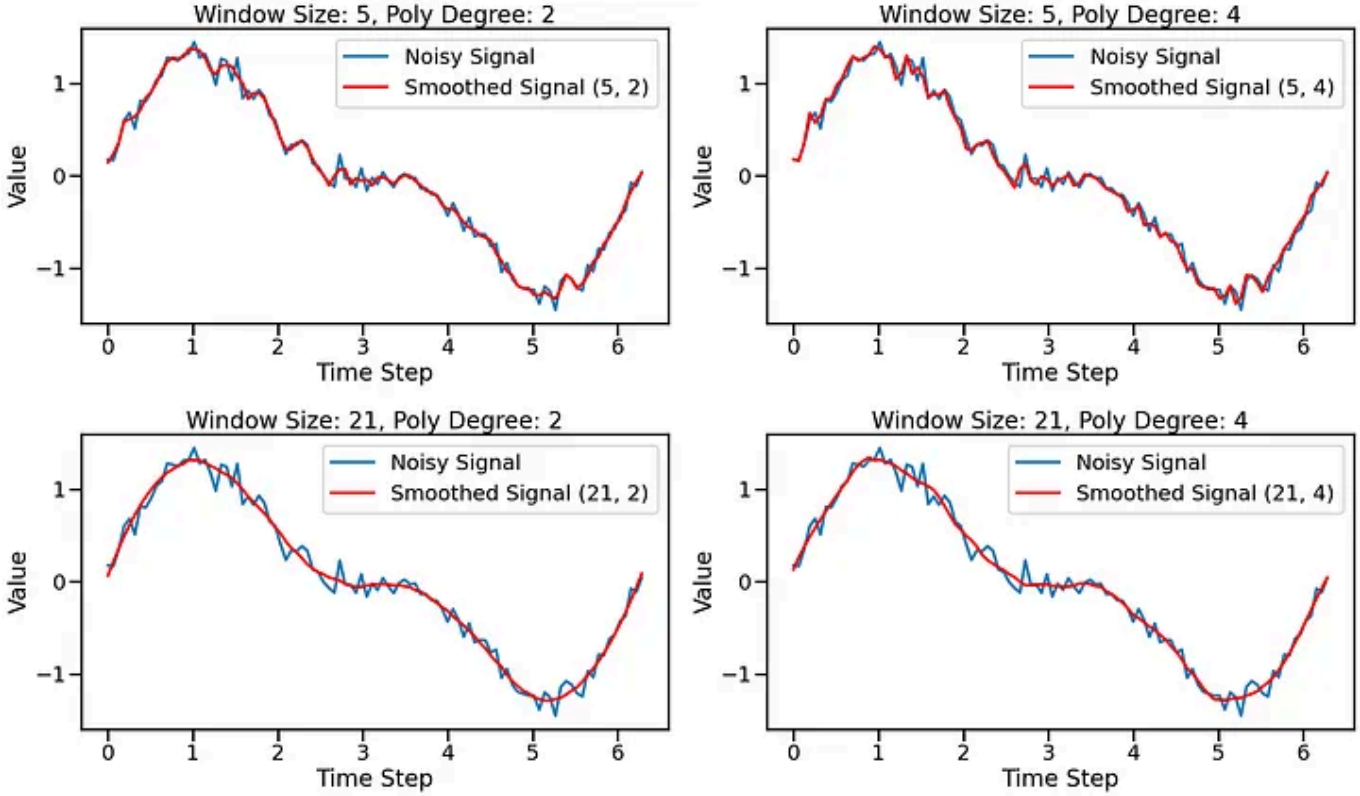
Step-by-step visualization of the filter in action

The smoothed signal retains the original sine wave shape while reducing the noise.

Exploring Different Window Sizes and Polynomial Degrees

```
fig, axs = plt.subplots(2, 2, figsize=(20, 12))

# Small window size, low polynomial degree
y_smooth_1 = savgol_filter(y_complex, 5, 2)
axs[0, 0].plot(x, y_complex, label='Noisy Signal')
axs[0, 0].plot(x, y_smooth_1, label='Smoothed Signal (5, 2)', color='red')
```

```
axs[0, 0].legend()
axs[0, 0].set_title('Window Size: 5, Poly Degree: 2')
plt.xlabel('Time Step')
plt.ylabel("Value")
plt.legend()
# Small window size, high polynomial degree
y_smooth_2 = savgol_filter(y_complex, 5, 4)
axs[0, 1].plot(x, y_complex, label='Noisy Signal')
axs[0, 1].plot(x, y_smooth_2, label='Smoothed Signal (5, 4)', color='red')
axs[0, 1].legend()
axs[0, 1].set_title('Window Size: 5, Poly Degree: 4')

# Large window size, low polynomial degree
y_smooth_3 = savgol_filter(y_complex, 21, 2)
axs[1, 0].plot(x, y_complex, label='Noisy Signal')
axs[1, 0].plot(x, y_smooth_3, label='Smoothed Signal (21, 2)', color='red')
axs[1, 0].legend()
axs[1, 0].set_title('Window Size: 21, Poly Degree: 2')

# Large window size, high polynomial degree
y_smooth_4 = savgol_filter(y_complex, 21, 4)
axs[1, 1].plot(x, y_complex, label='Noisy Signal')
axs[1, 1].plot(x, y_smooth_4, label='Smoothed Signal (21, 4)', color='red')
axs[1, 1].legend()
axs[1, 1].set_title('Window Size: 21, Poly Degree: 4')

plt.tight_layout()
plt.show()
```



Observations

- **Small Window Size, Low Polynomial Degree:** The filter smooths the data but may not capture the overall trend well, especially for signals with higher frequency components.
- **Small Window Size, High Polynomial Degree:** The filter can capture more complex trends but may overfit the noise, leading to less effective smoothing.
- **Large Window Size, Low Polynomial Degree:** The filter provides a stable smoothing effect but may smooth out important features of the signal.
- **Large Window Size, High Polynomial Degree:** The filter captures complex trends while providing effective smoothing but may introduce artifacts if the window size is too large relative to the signal's frequency components.

Practical Considerations

When using the Savitzky-Golay filter, choosing the appropriate window size and polynomial degree is crucial to achieving effective smoothing without distorting the signal. Here I will outline how to select these parameters, the trade-offs involved, and some limitations and potential pitfalls of the filter.

Choosing the Window Size and Polynomial Degree

1. **Window Size (2m+1):**

- **Small Window Size:** A smaller window size captures more local details but may not smooth the noise effectively. It is suitable for signals with high-frequency components or rapid changes.
- **Large Window Size:** A larger window size provides a more stable smoothing effect but can smooth out important features, especially in signals with rapid changes. It is suitable for signals with low-frequency components or slow variations.

2. Polynomial Degree (p):

- **Low Polynomial Degree:** A low-degree polynomial (e.g., p=2 or p=3) provides a simpler model that captures the general trend but may not fit complex patterns well.
- **High Polynomial Degree:** A high-degree polynomial (e.g., p=4 or p=5) can capture more complex trends but may overfit the noise, especially with smaller window sizes.

Some Trade-offs and “Rule of thumb”s

- **Balance Smoothing and Feature Preservation:** The goal is to smooth the noise while preserving the important features of the signal. This requires a balance between the window size and polynomial degree. Experiment

with different values to find the optimal parameters for your specific data.

- **Avoid Overfitting:** Using a high-degree polynomial with a small window size can lead to overfitting, where the polynomial captures the noise instead of the underlying trend. This results in a smoothed signal that still contains noise.
- **Signal Characteristics:** Consider the characteristics of your signal when choosing the parameters. For example, a signal with rapid changes requires a smaller window size and potentially a higher polynomial degree, while a slowly varying signal can benefit from a larger window size and a lower polynomial degree.

Limitations and Potential Pitfalls

1. **"Edge Effect":** The Savitzky-Golay filter has difficulty smoothing data points near the edges of the signal because there are fewer points available for fitting the polynomial. This can lead to less accurate smoothing at the boundaries.
2. **Assumption of Evenly Spaced Data:** The filter assumes that the data points are **evenly spaced** (and this is critical). If your data is unevenly spaced, you may need to preprocess it to ensure even spacing or use alternative methods that can handle uneven spacing.
3. **Choice of Parameters:** The effectiveness of the filter heavily depends on the chosen window size and polynomial degree. Incorrect choices can lead to poor smoothing performance, either by not reducing enough noise or by distorting the signal.
4. **Computational Complexity:** For very large datasets, the polynomial fitting process for each window can be computationally intensive.

Efficient implementation and optimization techniques may be necessary to handle large-scale data.

Practical Tips

1. **Start with Default Values:** A good starting point is to use a window size of 11 and a polynomial degree of 3. These values often provide a good balance between smoothing and feature preservation.
2. **Visual Inspection:** After applying the filter, visually inspect the smoothed signal to ensure that it effectively reduces noise while preserving important features.
3. **Parameter Tuning:** Experiment with different window sizes and polynomial degrees to find the optimal parameters for your specific data. Use cross-validation or other techniques to objectively evaluate the performance of different parameter choices.
4. **Edge Handling:** Consider using padding or extending the data at the edges to mitigate edge effects. Alternatively, use a different smoothing technique for the edges.

Machine Learning | Signal Processing | Time Series Analysis | Data Science | Mathematics

Written by Thomas Konstantinovsky

PhD Student & Data Scientist driven by curiosity—on a relentless quest to uncover the hidden patterns that shape our world.